

Mammoth Growth — Claude Code Tools Reference

All MCP tools, slash commands, and skills available in this project session

Project: mammoth-test · Engineer: Stan Kidd · Generated: 2026-05-25 · Model: claude-sonnet-4-6

MCP TOOLS

Server: **dbt** · Status: ✓ Connected · dbt=1.11.11, Snowflake adapter=1.11.5 · 47 tools total

CORE EXECUTION

TOOL	DESCRIPTION
mcp__dbt__build	Run models, tests, snapshots, and seeds in DAG order. Supports node selection, full-refresh, and vars.
mcp__dbt__run	Execute compiled SQL model files against the target database. Materializes models per their configured strategy.
mcp__dbt__test	Run data tests and unit tests. Supports node selection to scope to specific models or tags.
mcp__dbt__show	Execute arbitrary SQL against the warehouse and return results. Primary tool for data inspection and profiling.
mcp__dbt__compile	Generate executable SQL from model, test, and analysis files without running them. Useful for inspecting Jinja output.
mcp__dbt__parse	Parse and validate all project YAML and Jinja. Catches syntax errors before a build attempt.
mcp__dbt__clone	Clone models from one environment to another using zero-copy cloning where supported.
mcp__dbt__docs	Generate the project's documentation website from model descriptions and schema definitions.

PROJECT EXPLORATION

TOOL	DESCRIPTION
mcp__dbt__list	List project resources — models, tests, seeds, snapshots, sources — with optional type and selector filters.
mcp__dbt__get_all_models	Return all models with full metadata (materialization, tags, description, columns).
mcp__dbt__get_all_sources	Return all source definitions with database, schema, and table metadata.
mcp__dbt__get_all_macros	List all macros available in the project and installed packages.
mcp__dbt__get_model_details	Full metadata for a single model — columns, tests, config, compiled SQL.
mcp__dbt__get_model_parents	Upstream dependencies (parents) of a model in the DAG.
mcp__dbt__get_model_children	Downstream dependents (children) of a model in the DAG.
mcp__dbt__get_model_health	Test pass/fail status and freshness health for a model.
mcp__dbt__get_model_performance	Runtime performance statistics for a model across recent runs.
mcp__dbt__get_lineage	Full DAG lineage view — upstream and downstream — for a node.
mcp__dbt__get_lineage_dev	Lineage view scoped to the dev environment.
mcp__dbt__get_node_details_dev	Node details (config, SQL, tests) in the dev environment context.
mcp__dbt__get_source_details	Full metadata for a specific source table — freshness, columns, tests.
mcp__dbt__get_macro_details	Arguments, body, and usage information for a specific macro.
mcp__dbt__get_test_details	Configuration and failure details for a specific test node.

<code>mcp__dbt__get_seed_details</code>	Metadata and column definitions for a seed file.
<code>mcp__dbt__get_snapshot_details</code>	Strategy, unique key, and column metadata for a snapshot.
<code>mcp__dbt__get_exposure_details</code>	Owner, type, and upstream dependencies for a specific exposure.
<code>mcp__dbt__get_exposures</code>	List all exposures defined in the project.
<code>mcp__dbt__get_mart_models</code>	List mart (gold) models — the business-facing output layer.

SEMANTIC LAYER & METRICS

TOOL	DESCRIPTION
<code>mcp__dbt__get_dimensions</code>	List all semantic dimensions defined in the project's semantic models.
<code>mcp__dbt__get_entities</code>	List all semantic entities (join keys) for the semantic layer.
<code>mcp__dbt__get_semantic_model_details</code>	Full detail for a specific semantic model — measures, dimensions, entities.
<code>mcp__dbt__list_metrics</code>	List all metrics defined in the project.
<code>mcp__dbt__list_saved_queries</code>	List all saved queries available in the semantic layer.
<code>mcp__dbt__get_metrics_compiled_sql</code>	Return the compiled SQL that backs a specific metric definition.
<code>mcp__dbt__query_metrics</code>	Query metric values directly via the semantic layer.

DBT CLOUD ORCHESTRATION

TOOL	DESCRIPTION
<code>mcp__dbt__list_jobs</code>	List all dbt Cloud jobs in the connected project.
<code>mcp__dbt__list_jobs_runs</code>	List run history for dbt Cloud jobs.
<code>mcp__dbt__get_job_details</code>	Configuration and schedule details for a specific dbt Cloud job.
<code>mcp__dbt__get_job_run_details</code>	Status, timing, and step details for a specific job run.
<code>mcp__dbt__get_job_run_error</code>	Error message, failing node, and stack trace for a failed run.
<code>mcp__dbt__list_job_run_artifacts</code>	List artifacts (manifest, run_results) produced by a job run.
<code>mcp__dbt__trigger_job_run</code>	Trigger a dbt Cloud job run on demand.
<code>mcp__dbt__retry_job_run</code>	Retry a failed job run from the point of failure.
<code>mcp__dbt__cancel_job_run</code>	Cancel a currently running dbt Cloud job.
<code>mcp__dbt__list_projects</code>	List all dbt Cloud projects accessible in the connected account.

DOCUMENTATION SEARCH

TOOL	DESCRIPTION
<code>mcp__dbt__get_product_doc_pages</code>	Fetch specific pages from the official dbt product documentation.
<code>mcp__dbt__search_product_docs</code>	Full-text search across dbt product documentation.

CUSTOM SLASH COMMANDS

Defined in `.cClaude/commands/` · 29 commands total

SESSION MANAGEMENT

COMMAND	DESCRIPTION
<code>/all_tools</code>	List every MCP tool, slash command, and skill available in the current Claude Code session.
<code>/save-context</code>	Save current session context, progress, and decisions to a file for future reference or handoff.
<code>/clock-out</code>	End a work session cleanly — save context, commit any WIP, and write a handoff note.
<code>/load_ai_docs</code>	Conditionally load AI/LLM documentation files relevant to the current task.
<code>/read_conditional_docs</code>	Load documentation files based on what the current task actually requires.

GIT & GITHUB

COMMAND	DESCRIPTION
<code>/git-create-branch</code>	Create a new branch following Mammoth naming conventions (sk/feature-name).
<code>/git-commit</code>	Stage and commit specific files with a descriptive, conventional commit message.
<code>/git-commit-all</code>	Stage all changed files and commit with a descriptive message.
<code>/github-create-pr</code>	Create a GitHub pull request using the full Mammoth Growth PR template with context summary.
<code>/pr-from-spec</code>	Build a complete PR from a finished technical specification — branch, commits, and PR description.

DBT BUILD COMMANDS

COMMAND	DESCRIPTION
<code>/build</code>	Run <code>dbt build</code> on a specific model or node selector. Validates model before marking work done.
<code>/build-bronze-models</code>	Build all bronze (raw parse) models defined in a tech spec using the dbt MCP server.
<code>/build-silver-models</code>	Build all silver (clean and conform) models defined in a tech spec.
<code>/build-gold-models</code>	Build all gold (metrics and aggregations) models defined in a tech spec.
<code>/build-full-spec</code>	Build all three layers (bronze → silver → gold) from a tech spec in a single agentic run.

DATA PROFILING

COMMAND	DESCRIPTION
<code>/data-profile</code>	Profile source tables — row counts, null rates, distinct values, and data samples.
<code>/profile_table</code>	Full Level 1–4 profile of a single table: shape, grain, nulls, dates, numerics, categoricals.
<code>/profile_column</code>	Deep profile of a single column — distribution, null rate, cardinality, string lengths.
<code>/profile_column_relationship</code>	Profile the join quality between two columns — match rate, cardinality, fan-out risk.
<code>/profile_domain</code>	Profile all distinct values in a categorical column to generate <code>accepted_values</code> test lists.
<code>/profile_table_relationships</code>	Profile all FK relationships between tables defined in a tech spec.
<code>/profile_plan</code>	Generate a structured profiling plan for all source tables in a tech spec before building begins.

PLANNING & SPECIFICATION

COMMAND	DESCRIPTION
<code>/tech_spec_plan</code>	Convert a Business Requirements Document (BRD) into a full Technical Specification with model designs.
<code>/validate-spec</code>	Validate a technical specification — check completeness, consistency, and build-readiness.
<code>/bug_plan</code>	Diagnose a failing dbt model or test and create a structured fix plan.
<code>/chore_plan</code>	Plan a maintenance or chore task (refactor, syntax update, dependency upgrade).
<code>/metaprompt_workflow</code>	Generate an optimized Claude prompt for a complex agentic task using meta-prompting techniques.

SKILLS

Defined in `.claude/skills/` · Loaded via the Skill tool when a matching command is invoked · 10 skills total

SKILL	CATEGORY	DESCRIPTION
<code>data-profiling</code>	Data Quality	Four-level profiling framework: (1) Table shape & grain, (2) Column distributions, (3) Relationship/join quality, (4) Data quality flags. Produces structured TABLE PROFILE REPORT output with recommended tests.
<code>dbt-best-practices</code>	Standards	SQL and YAML standards for this project — keyword casing, field naming, CTE preference, ref() usage, materialization rules per layer, and test requirements by column type.
<code>dbt-planning-router</code>	Standards	Routes incoming planning requests to the correct skill (tech-spec-plan, build-from-spec, data-profiling) based on task type and current project state.
<code>dbt-implementation-validator</code>	Standards	Validates a completed dbt implementation against its technical specification — checks model existence, column alignment, test coverage, and materialization correctness.
<code>build-from-spec</code>	Build	Orchestrates the full build of a layer (bronze, silver, or gold) from a technical specification. Reads spec → writes SQL → writes schema YAML → runs dbt_build → validates.
<code>tech-spec-plan</code>	Planning	Converts a Business Requirements Document into a full Technical Specification: source mapping, model designs per layer, column definitions, join logic, test recommendations, and grain documentation.
<code>validate-tech-spec</code>	Planning	Reviews a technical spec for completeness and build-readiness — checks that all sources are profiled, grains are defined, joins are validated, and tests are specified.
<code>pr-from-spec</code>	Build	Builds a complete GitHub PR from a finished spec — creates branch, stages all model and schema files, writes a conventional commit, and opens the PR with a full context summary.
<code>git</code>	Utility	Git workflow operations following Mammoth conventions: branch naming, commit formatting, and PR creation rules.
<code>meta-skill</code>	Utility	Skill composition and orchestration — used internally to chain multiple skills together for complex multi-step tasks.

BUILT-IN CLAUDE CODE COMMANDS

Native Claude Code CLI commands — available in every session regardless of project configuration

COMMAND	DESCRIPTION	COMMAND	DESCRIPTION
<code>/clear</code>	Clear the conversation history	<code>/login</code>	Log in to Claude Code
<code>/compact</code>	Compact conversation to save context window	<code>/logout</code>	Log out of Claude Code
<code>/config</code>	View or change Claude Code configuration	<code>/memory</code>	Edit Claude memory files
<code>/cost</code>	Show token usage and cost for this session	<code>/model</code>	Switch the active Claude model
<code>/doctor</code>	Check Claude Code installation health	<code>/permissions</code>	View and manage tool permissions
<code>/help</code>	Get help with Claude Code features	<code>/pr-comments</code>	Fetch comments on a GitHub PR
		<code>/status</code>	Show account and session status
		<code>/terminal-setup</code>	Set up shell integration

PROJECT QUICK REFERENCE

From CLAUDE.md — current project configuration at time of generation

DATA SOURCES

CLIENT	DATABASE / SCHEMA	SCALE	USE CASE	MODELS PATH
tpch_sf1000	SNOWFLAKE_SAMPLE_DATA / TPCH_SF1000	6B rows (LINEITEM)	Order analytics — revenue by segment, supplier nation, part type	models/tpch_sf1000/
tpch_sf10	SNOWFLAKE_SAMPLE_DATA / TPCH_SF10	60M rows (LINEITEM)	Supplier performance — delivery, discount rates, return rates	models/tpch_sf10/
pharma_sales	MAMMOTH_DB / MAMMOTH_SCHEMA_pharma_sales_seeds	180 rows (5 seeds)	Sales effectiveness — rep performance, physician scorecard, product performance	models/pharma_sales/
champion_homes	TBD (pending client access)	TBD	Enterprise performance management — 9 business functions	models/champion_homes/

LAYER RULES

LAYER	MATERIALIZATION	RULE
Bronze	view	No business logic. Parse and rename raw source columns only. Use source() or ref('seed_*').
Silver	table	Clean, conform, join. Apply business rules and filters. Use ref() to bronze models.
Gold	table	Business metrics and aggregations. Mart-ready. Use ref() to silver models only.

SQL STANDARDS

RULE	DETAIL
Keywords	ALL SQL keywords UPPERCASE (SELECT, FROM, WHERE, JOIN, GROUP BY, etc.)
Field names	all_lowercase_with_underscores
Table aliases	Always alias with a meaningful short name (e.g. o for orders, li for lineitem)
Query structure	CTEs preferred over subqueries
Model references	Always use ref() — never hardcode schema.table names in model SQL
Branch naming	sk/feature-name
Schema prefix (dev)	MAMMOTH_SCHEMA_ (e.g. MAMMOTH_SCHEMA_tpch_sf10_bronze)